FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 2

Name: Aufwand in h:

Mat.Nr: Punkte:

Übungsgruppe: korrigiert:

Beispiel 1 (24 Punkte) Adressenverwaltung: Eine Software für eine Adressenverwaltung (Adressen und Personen) soll ihre Daten aus Dateien einlesen, Adressen mit Personen richtig verlinken und anschließend in unterschiedlichen Dateiformaten (Ascii und Html) ausgeben.

Die Daten von Personen (Vorname, Nachname, Verweis auf zugehörige Adresse) können in einer oder mehreren Dateien abgelegt sein. Eine Personendatei ist folgendermaßen aufgebaut:

```
##########################
# Datei: pers.txt
# erstellt am: 01.10.2022
##########################
Donald Duck 0
Dagobert Duck 1
Daniel Düsentrieb 2
Max Möchtegern 2
Brid Patt 0
Jack Unterweger 1
Frank Gust 2
...
```

Die Header-Information kann überlesen werden. Die Nummer nach dem Namen entspricht dem Index auf die zugehörige Adresse.

Die Adressen sind in einer einzelnen Datei mit folgendem Aufbau abgelegt:

```
#########################################################
# Datei: adr.txt
# erstellt am: 01.10.2022
# Beschreibung: Datei speichert die Adressen von Personen
#########################################################
Kellergasse 3 4287 Radburg
Diemstrasse 1 2345 Hinterhof
Waldstrasse 45 6785 Burghausen
Domweg 123 9865 Mixbergen
...
```

Die Header-Information kann überlesen werden und die Reihenfolge der einzelnen Adressen entspricht dem Index-Verweis in der Personendatei. Die Adressen und Personen aus den einzelnen Dateien sind einzulesen und in einer entsprechenden Datenstruktur abzulegen. Die Datenstruktur verknüpft die Adressen mit den zugehörigen Personen. Die Einlesereihenfolge der Dateien ist beliebig.

Die Ausgabe der Adressen mit Personen kann entweder in einer Ascii- oder in einer HTML-Datei erfolgen:

```
Adressen mit zugeordneten Personen
----------------------------------
4287 Radburg
Kellergasse 3
    Donald Duck
    Brid Patt


2345 Hinterhof
Diemstrasse 1
    Dagobert Duck
    Jack Unterweger


6785 Burghausen
Waldstrasse 45
    Daniel Düsentrieb
    Max Möchtegern
    Frank Gust


9865 Mixbergen
Domweg 123
```

Adressen mit zugeordneten Personen

4287 Radburg
Kellergasse 3
Donald Duck
Brid Patt

2345 Hinterhof
Diemstrasse 1
Dagobert Duck
Jack Unterweger

6785 Burghausen
Waldstrasse 45
Daniel Düsentrieb
Max Möchtegern
Frank Gust

9865 Mixbergen
Domweg 123

Entwerfen Sie für die gesammte Adressenverwaltung ein Klassendiagramm und implementieren Sie die einzelnen Klassen. Verwenden Sie an geeigneter Stelle das Design-Pattern *Template Method*. Achten Sie zusätzlich auf eine entsprechende Fehlerbehandlung!

Eine Anwendung der Adressenverwaltung durch den Benutzer sieht folgendermaßen aus:

```
1  int main()
2  {
3     AddressManager adrMan;
4     PersonReader persReader;
5     AddressReader adrReader;
6     AsciiWriter ascWriter;
7     HtmlWriter  htmWriter;
8
9     adrMan.ReadFile("pers.txt", &persReader);
10    adrMan.ReadFile("pers2.txt", &persReader);
11    adrMan.ReadFile("adr.txt", &adrReader);
12    adrMan.ReadFile("pers3.txt", &persReader);
13
14    adrMan.WriteAddresses("addresses.txt", &ascWriter);
15    adrMan.WriteAddresses("addresses.html", &htmWriter);
16
17    return 0;
18 }
```

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und verse-

hen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!

HSD
OBERÖSTERREICH
FH-HAGENBERG

Systemdokumentation

Adressverwaltung

Version 1.0

J. Rieger, L. Kieweg

Hagenberg, 21. Oktober 2025

Dieses Dokument zeigt den prinzipiellen Aufbau einer Systemdokumentation für Software-Projekte. Die einzelnen Kapitel sind mit Kommentaren versehen, welche die Struktur und den Inhalt erläutern.

Inhaltsverzeichnis

1 Organisatorisches **2**
1.1 Team . . . 2
1.2 Aufteilung der Verantwortungsbereiche . . . 2
1.3 Aufwand . . . 2

2 Anforderungsdefinition (Systemspezifikation) **2**

3 Systementwurf **3**
3.1 Klassendiagram . . . 3
3.2 Designentscheidungen . . . 3

4 Komponentenentwurf **4**

5 Testprotokollierung **4**
5.1 Ausgabe Konsole Testtreiber . . . 4
5.2 Ascii Output File . . . 6
5.3 Html Output File . . . 7

6 Quellcode **8**

1 Organisatorisches

1.1 Team

- Julian Rieger, Matr.-Nr: S2510306023, E-Mail: S2510306023@fhooe.at
- Leander Kieweg, Matr.-Nr: S2510306014, E-Mail: S2510306014@fhooe.at

1.2 Aufteilung der Verantwortungsbereiche

- Julian Rieger
  - Implementierung der Klassen: Address, Person, Reader, AddressReader, PersonReader
  - Implementierung des Testtreibers
  - Verfassen der Systemdokumentation
- Leander Kieweg
  - Implementierung der Klassen: AddressManager, HtmlWriter, AsciiWriter, Writer, Object, IReadOnlyAddressManager, IWriteOnlyAddressManager
  - Design des Klassendiagrams
  - Implementierung des Testtreibers

1.3 Aufwand

- Julian Rieger: geschätzt: 7h, tatsächlich 11h
- Leander Kieweg: geschätzt 7h, tatsächlich 11h

2 Anforderungsdefinition (Systemspezifikation)

Es wird eine Software zur Verwaltung von Adressen und Personen entwickelt. Sie liest Personen und Adressen aus Textdateien ein, verknüpft jede Person mit ihrer Adresse und gibt alle Adressen samt zugeordneter Personen im ASCII- oder HTML-Format aus.

Funktionale Anforderungen

- **Personendatei:** eine Zeile pro Person im Format `Vorname Nachname Index`. Der Index (0-basiert) verweist auf die Position der Adresse in der Adressdatei. Personen können auf mehrere Dateien verteilt sein.
- **Adressdatei:** eine Zeile pro Adresse im Format `Strasse Hausnummer PLZ Ort`. Die Zeilenreihenfolge ergibt den Index.
- Zeilen, die mit `#` beginnen (Header), sowie Leerzeilen werden ignoriert.
- Die Reihenfolge, in der Dateien eingelesen werden, ist beliebig.

- Ausgabe: je Adresse PLZ und Ort, Strasse und Hausnummer, danach alle zugehörigen Personen; wahlweise als ASCII- oder HTML-Datei.
- Anlegen von Personen und Adressen erfolgt ausschließlich durch die Reader; der `AddressManager` bietet keinen öffentlichen Schreibzugriff.

Fehlerbehandlung

Fehler werden über Exceptions gemeldet: `std::invalid_argument` (Null-Zeiger für Reader/Writer), `std::runtime_error` (Datei nicht les-/schreibbar, falsches Zeilenformat) und `std::out_of_range` (ungültiger Index, Person verweist auf nicht vorhandene Adresse).

3 Systementwurf

3.1 Klassendiagram

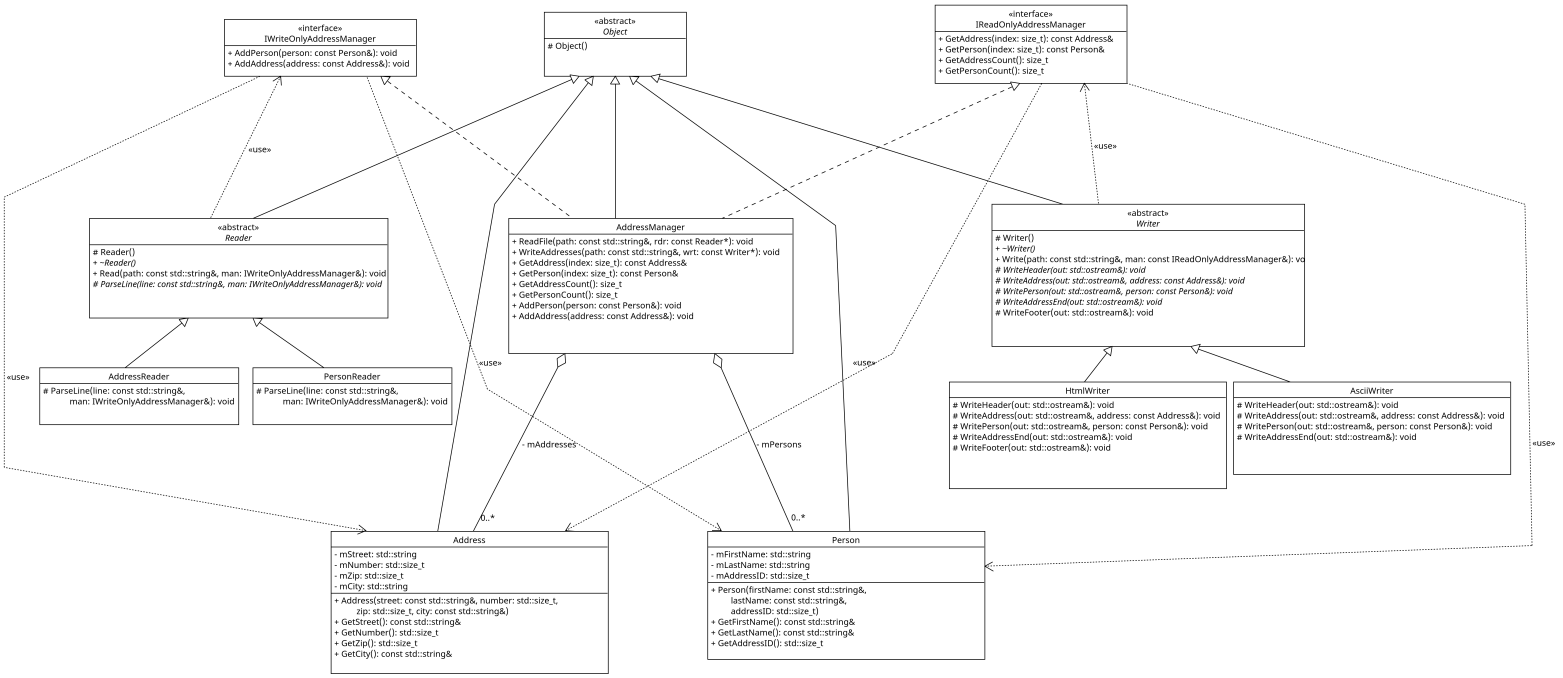


Abbildung 1: Klassendiagramm AddressManager

Anmerkung: Das Klassendiagram steht für bessere Lesbarkeit ebenfalls im Sysdoku-Ordner zur Verfügung.

3.2 Designentscheidungen

Das Template Method Pattern wird für Reader und Writer eingesetzt. `Reader::Read()` legt den Ablauf fest (Datei öffnen, Header und Leerzeilen überspringen, jede Datenzeile an `ParseLine()` übergeben); `AddressReader` und `PersonReader` implementieren nur

`ParseLine()`. `Writer::Write()` legt die Reihenfolge Header → je Adresse (Adresse, Personen, Adressende) → Footer fest; `HtmlWriter` und `AsciiWriter` implementieren die Schritte `WriteHeader`, `WriteAddress`, `WritePerson` und `WriteAddressEnd`. Diese sind pure virtual; `WriteFooter` ist ein Hook mit leerer Standardimplementierung, den nur der `HtmlWriter` überschreibt. Reader und Writer sind abstrakt (protected Konstruktor) und zustandslos, daher sind alle Methoden `const`.

Interfaces. Reader kennen nur `IWriteOnlyAddressManager`, Writer nur `IReadOnlyAddressManager`. Der `AddressManager` erbt von `IWriteOnlyAddressManager` privat, daher kann niemand von außen Daten einfügen. Alle Klassen außer den Interfaces erben direkt oder indirekt von `Object`.

Verknüpfung über Index. Personen speichern den Index ihrer Adresse statt eines Zeigers. Dadurch ist die Einlesereihenfolge beliebig, und es entstehen keine hängenden Zeiger beim Wachsen der `std::vector`-Container.

Prüfung beim Schreiben. Ob ein Adressindex gültig ist, lässt sich erst prüfen, wenn alle Dateien gelesen sind. `Writer::Write()` prüft deshalb vor dem Öffnen der Ausgabedatei alle Personen und wirft `std::out_of_range`, sodass keine halb geschriebene Datei entsteht.

Value Classes. `Person` und `Address` sind unveränderlich (Werte nur im Konstruktor gesetzt).

Fehlerstrategie. Fehler werden als Exceptions gemeldet. Der `Reader` ergänzt Dateiname und Zeilennummer. Reader und Writer werden als Zeiger übergeben (Vorgabe der Schnittstelle), der `AddressManager` übernimmt keinen Besitz und prüft auf `nullptr`.

4 Komponentenentwurf

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

5.1 Ausgabe Konsole Testtreiber

```
[SUCCESS] person and address file are read
[SUCCESS] persons and addresses are stored
[SUCCESS] first person is Donald Duck with address 0
[SUCCESS] first address is Kellergasse 3, 4287 Radburg
[SUCCESS] ascii and html file are written
[SUCCESS] null reader -> invalid_argument
[SUCCESS] null writer -> invalid_argument
[SUCCESS] missing file -> runtime_error
[SUCCESS] wrong line format -> runtime_error
```

[SUCCESS] wrong address index -> out_of_range
[SUCCESS] wrong person index -> out_of_range
[SUCCESS] person without address -> out_of_range

5.2 Ascii Output File

Adressen mit zugeordneten Personen

4287 Radburg
Kellergasse 3
    Donald Duck
    Brid Patt

2345 Hinterhof
Diemstrasse 1
    Dagobert Duck
    Jack Unterweger

6785 Burghausen
Waldstrasse 45
    Daniel Düsentrieb
    Max Möchtegern
    Frank Gust

9865 Mixbergen
Domweg 123

5.3 Html Output File

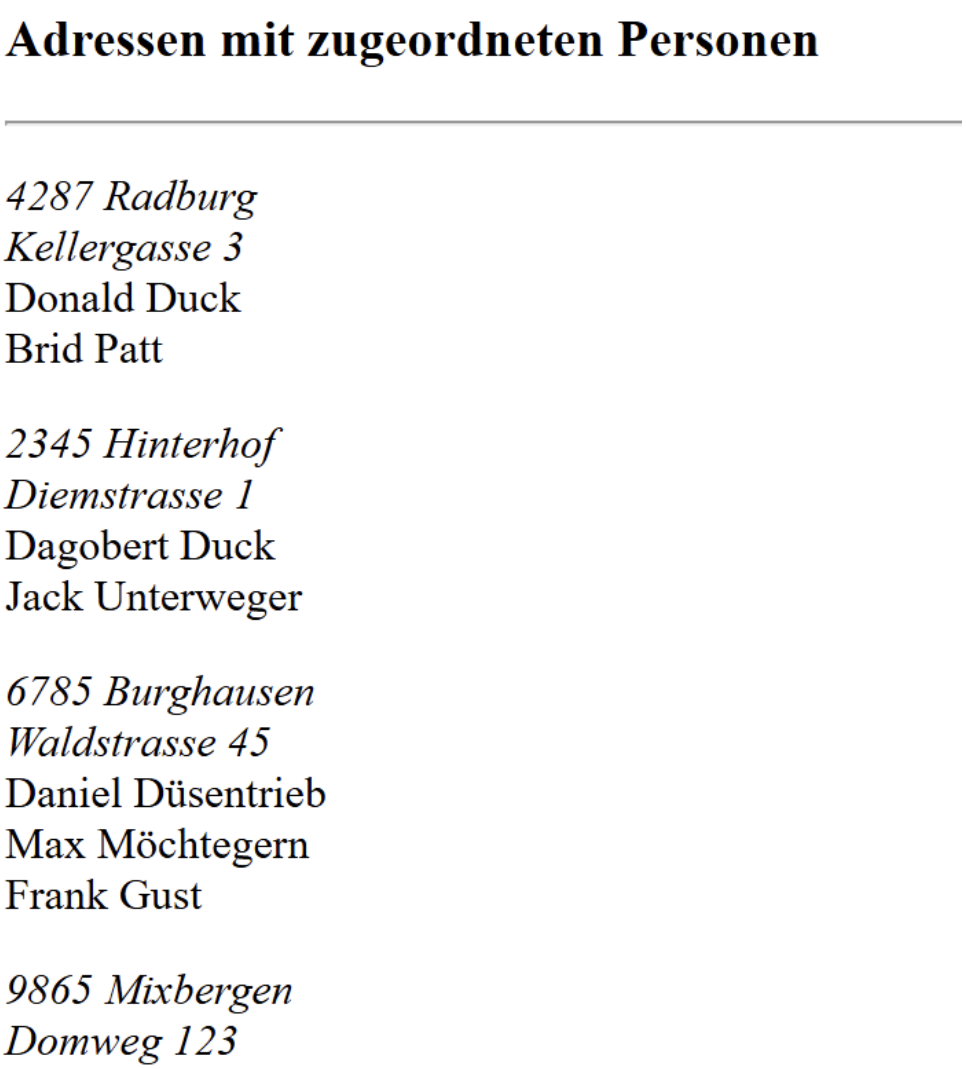


Abbildung 2: HtmlOutput

6 Quellcode

Listing 1: ../Object.h

```
#ifndef OBJECT_H
#define OBJECT_H
#include <string>

/// @author Leander Kieweg
/// @date 21.10.2026
/// @brief Base class of all base classes.
///
/// Is the common root class of all classes. If there is no other base class,
/// this class is to be selected as the base class.
class Object
{
public:
   /// @brief Virtual default destructor -> once virtual, always virtual.
   ///
   /// It is not necessary to declare another virtual destructor
   /// in derived classes.
   virtual ~Object() = default;

protected:
   /// @brief Protected default constructor that makes the class abstract.
   Object() = default;
};

#endif
```

Listing 2: ../IReadOnlyAddressManager.h

```
#ifndef IREADONLYADDRESSMANAGER_H
#define IREADONLYADDRESSMANAGER_H

#include <cstddef>
#include "Address.h"
#include "Person.h"

/// @author Leander Kieweg
/// @date 21.10.2026
/// @brief Interface for read-only access to persons and addresses.
///
/// Follows the Interface Segregation Principle (ISP). It provides only
/// inspection methods required by Writers to format and export data,
/// preventing unintended modifications.
class IReadOnlyAddressManager
{
public:
    /// @brief Virtual default destructor for safe polymorphic deletion.
    virtual ~IReadOnlyAddressManager() = default;

    /// @brief Access function for an address.
    /// @param[in] index is the position of the address.
    /// @returns a constant reference to the address.
    /// @throws std::out_of_range if the index does not exist.
    virtual const Address& GetAddress(std::size_t index) const = 0;

    /// @brief Access function for a person.
    /// @param[in] index is the position of the person.
    /// @returns a constant reference to the person.
    /// @throws std::out_of_range if the index does not exist.
    virtual const Person& GetPerson(std::size_t index) const = 0;

    /// @brief Access function for the number of addresses.
    /// @returns the number of stored addresses.
    virtual std::size_t GetAddressCount() const = 0;

```

```
37      /// @brief Access function for the number of persons.
38      /// @returns the number of stored persons.
39      virtual std::size_t GetPersonCount() const = 0;
40  };
41
42  #endif // IREADONLYADDRESSMANAGER_H
```

Listing 3: ../IWriteOnlyAddressManager.h

```
1   #ifndef IWRITEONLYADDRESSMANAGER_H
2   #define IWRITEONLYADDRESSMANAGER_H
3
4   #include "Address.h"
5   #include "Person.h"
6
7   /// @author Leander Kieweg
8   /// @date 21.10.2026
9   /// @brief Interface for adding persons and addresses (write-only access).
10  ///
11  /// Follows the Interface Segregation Principle (ISP). It provides only
12  /// mutating methods required by Readers to populate data, without exposing
13  /// read access or management functions.
14  class IWriteOnlyAddressManager
15  {
16  public:
17      /// @brief Virtual default destructor for safe polymorphic deletion.
18      virtual ~IWriteOnlyAddressManager() = default;
19
20      /// @brief Adds a person to the manager.
21      /// @param[in] person is the person to store.
22      virtual void AddPerson(const Person& person) = 0;
23
24      /// @brief Adds an address to the manager.
25      /// @param[in] address is the address to store.
26      virtual void AddAddress(const Address& address) = 0;
27  };
28
29  #endif // IWRITEONLYADDRESSMANAGER_H
```

Listing 4: ../Address.h

```
1   #ifndef ADDRESS_H
2   #define ADDRESS_H
3   #include <cstddef>
4   #include <string>
5   #include "Object.h"
6
7   /// @author Julian Rieger
8   /// @date 21.10.2026
9   /// @brief Stores a postal address (street, house number, zip code, city).
10  ///
11  /// The class is a simple value class. All data is set in the constructor
12  /// and can only be read afterwards, so an address never changes.
13  class Address : public Object {
14  public:
15      /// @brief Creates an address.
16      /// @param[in] street is the name of the street.
17      /// @param[in] number is the house number.
18      /// @param[in] zip is the zip code.
19      /// @param[in] city is the name of the city.
20      Address(const std::string& street, std::size_t number, std::size_t zip, const
            std::string& city);
21
22      /// @brief Access function for the street.
23      /// @returns a constant reference to the name of the street.
24      const std::string& GetStreet() const;
25
```

```
26      /// @brief Access function for the house number.
27      /// @returns the house number.
28      std::size_t GetNumber() const;
29
30      /// @brief Access function for the zip code.
31      /// @returns the zip code.
32      std::size_t GetZip() const;
33
34      /// @brief Access function for the city.
35      /// @returns a constant reference to the name of the city.
36      const std::string& GetCity() const;
37
38  private:
39      /// @brief Name of the street.
40      std::string mStreet;
41      /// @brief House number.
42      std::size_t mNumber;
43      /// @brief Zip code.
44      std::size_t mZip;
45      /// @brief Name of the city.
46      std::string mCity;
47  };
48
49  #endif // ADDRESS_H
```

Listing 5: ../Address.cpp

```
1   #include "Address.h"
2
3   Address::Address(const std::string& street, std::size_t number, std::size_t zip,
        const std::string& city)
4       : mStreet(street),
5       mNumber(number),
6       mZip(zip),
7       mCity(city) {
8   }
9
10  const std::string& Address::GetStreet() const {
11      return mStreet;
12  }
13
14  std::size_t Address::GetNumber() const {
15      return mNumber;
16  }
17
18  std::size_t Address::GetZip() const {
19      return mZip;
20  }
21
22  const std::string& Address::GetCity() const {
23      return mCity;
24  }
```

Listing 6: ../Person.h

```
1   #ifndef PERSON_H
2   #define PERSON_H
3   #include <cstddef>
4   #include <string>
5   #include "Object.h"
6
7   /// @author Julian Rieger
8   /// @date 21.10.2026
9   /// @brief Stores a person with its name and a link to its address.
10  ///
11  /// The link is the index of the address in the AddressManager. The person
12  /// does not know the address itself, so persons and addresses can be read
```

```
13 /// from files in any order.
14 class Person : public Object
15 {
16 public:
17     /// @brief Creates a person.
18     /// @param[in] firstName is the first name of the person.
19     /// @param[in] lastName is the last name of the person.
20     /// @param[in] addressID is the index of the address of the person.
21     Person(const std::string& firstName, const std::string& lastName, std::size_t
           addressID);
22
23     /// @brief Access function for the first name.
24     /// @returns a constant reference to the first name.
25     const std::string& GetFirstName() const;
26
27     /// @brief Access function for the last name.
28     /// @returns a constant reference to the last name.
29     const std::string& GetLastName() const;
30
31     /// @brief Access function for the address index.
32     /// @returns the index of the address of this person.
33     std::size_t GetAddressID() const;
34
35 private:
36     /// @brief First name of the person.
37     std::string mFirstName;
38     /// @brief Last name of the person.
39     std::string mLastName;
40     /// @brief Index of the address in the AddressManager.
41     std::size_t mAddressID;
42 };
43
44 #endif // PERSON_H
```

Listing 7: ../Person.cpp

```
1  #include "Person.h"
2
3  Person::Person(const std::string& firstName, const std::string& lastName, std::
       size_t addressID)
4      : mFirstName(firstName),
5      mLastName(lastName),
6      mAddressID(addressID) {
7  }
8
9  const std::string& Person::GetFirstName() const {
10     return mFirstName;
11 }
12
13 const std::string& Person::GetLastName() const {
14     return mLastName;
15 }
16
17 std::size_t Person::GetAddressID() const {
18     return mAddressID;
19 }
```

Listing 8: ../Reader.h

```
1 #ifndef READER_H
2 #define READER_H
3
4 #include <string>
5 #include "Object.h"
6 #include "IWriteOnlyAddressManager.h"
7
8 /// @author Julian Rieger
```

```
9  /// @date 21.10.2026
10 /// @brief Abstract base class for all readers.
11 ///
12 /// The class uses the design pattern Template Method. The method Read
13 /// defines the fixed procedure of reading a file: open the file, skip the
14 /// header and empty lines and pass every data line to ParseLine. The derived
15 /// classes only implement ParseLine and so decide about the line format.
16 class Reader : public Object
17 {
18 public:
19     /// @brief Reads a file and stores its content in the manager.
20     ///
21     /// This is the template method. Lines that start with '#' (header) and
22     /// lines that contain only white space are skipped. Every other line is
23     /// passed to ParseLine.
24     /// @param[in] path is the path of the file to read.
25     /// @param[in,out] man is the manager that stores the parsed data.
26     /// @throws std::runtime_error if the file cannot be opened or read, or
27     ///         if a line has a wrong format.
28     void Read(const std::string& path, IWriteOnlyAddressManager& man) const;
29
30 protected:
31     /// @brief Protected default constructor that makes the class abstract.
32     Reader() = default;
33
34     /// @brief Parses one data line and adds the result to the manager.
35     /// @param[in] line is the line to parse (without line break).
36     /// @param[in,out] man is the manager that stores the parsed data.
37     /// @throws std::runtime_error if the line has a wrong format.
38     virtual void ParseLine(const std::string& line, IWriteOnlyAddressManager& man)
           const = 0;
39 };
40
41 #endif // READER_H
```

Listing 9: ../Reader.cpp

```
1  #include "Reader.h"
2  #include <fstream>
3  #include <stdexcept>
4
5  void Reader::Read(const std::string& path, IWriteOnlyAddressManager& man) const
6  {
7      std::ifstream file(path);
8      if (!file.is_open()) {
9          throw std::runtime_error("Error in Reader: could not open file '" + path +
               "'");
10     }
11
12     std::string line;
13     while (std::getline(file, line))
14     {
15         // Skip the header (lines starting with '#') and empty lines
16         if (!std::isalpha(line[0]) || line[0] == '#')
17         {
18             continue;
19         }
20         ParseLine(line, man);
21     }
22
23     // getline also stops at a real read error, not only at the end of the file.
24     if (file.bad()) {
25         throw std::runtime_error("Error in Reader: could not read file '" + path +
               "'");
26     }
27
28     file.close();
29 }
```

Listing 10: ../AddressReader.h

```h
#ifndef ADDRESSREADER_H
#define ADDRESSREADER_H

#include "Reader.h"
#include <string>

/// @author Julian Rieger
/// @date 21.10.2026
/// @brief Reads addresses from a file.
///
/// Every data line has the format "street number zip city". The position
/// of the address in the file is the index that the persons refer to.
class AddressReader : public Reader
{
protected:
    /// @brief Parses a line of an address file and adds the address to the
        manager.
    /// @param[in] line is the line to parse.
    /// @param[in,out] man is the manager that stores the address.
    /// @throws std::runtime_error if the line is not "street number zip city".
    void ParseLine(const std::string& line, IWriteOnlyAddressManager& man) const
        override;
};

#endif //ADDRESSREADER_H
```

Listing 11: ../AddressReader.cpp

```cpp
#include "AddressReader.h"
#include <sstream>
#include <stdexcept>

void AddressReader::ParseLine(const std::string& line, IWriteOnlyAddressManager&
    man) const
{
    std::string street;
    std::size_t number;
    std::size_t zip;
    std::string city;

    std::stringstream sstream(line);

    if (sstream >> street >> number >> zip >> city)
    {
        Address a{ street, number, zip, city };
        man.AddAddress(a);
    }
    else
    {
        throw std::runtime_error("Error in AddressReader: bad stream");
    }
}
```

Listing 12: ../PersonReader.h

```h
#ifndef PERSONREADER_H
#define PERSONREADER_H

#include "Reader.h"
#include <string>

/// @author Julian Rieger
/// @date 21.10.2026
/// @brief Reads persons from a file.
///
/// Every data line has the format "firstname lastname addressIndex".
```

```
12 /// The parsed person is added to the AddressManager.
13 class PersonReader : public Reader
14 {
15 protected:
16     /// @brief Parses a line of a person file and adds the person to the manager.
17     /// @param[in] line is the line to parse.
18     /// @param[in,out] man is the manager that stores the person.
19     /// @throws std::runtime_error if the line is not "firstname lastname index".
20     void ParseLine(const std::string& line, IWriteOnlyAddressManager& man) const
           override;
21 };
22
23 #endif //PERSONREADER_H
```

Listing 13: ../PersonReader.cpp

```
1  #include "PersonReader.h"
2  #include <sstream>
3  #include <stdexcept>
4
5  void PersonReader::ParseLine(const std::string& line, IWriteOnlyAddressManager&
       man) const
6  {
7      std::string firstname;
8      std::string lastname;
9      std::size_t number;
10
11     std::stringstream sstream(line);
12
13     if (sstream >> firstname >> lastname >> number)
14     {
15         Person p{ firstname, lastname, number };
16         man.AddPerson(p);
17     }
18     else
19     {
20         throw std::runtime_error("Error in PersonReader: bad stream");
21     }
22 }
```

Listing 14: ../Writer.h

```
1  #ifndef WRITER_H
2  #define WRITER_H
3  #include <ostream>
4  #include <string>
5  #include "Address.h"
6  #include "Person.h"
7  #include "Object.h"
8  #include "IReadOnlyAddressManager.h"
9
10 /// @author Leander Kieweg
11 /// @date 19.10.2026
12 /// @brief Abstract base class for all writers.
13 ///
14 /// The class uses the design pattern Template Method. The method Write
15 /// defines the fixed order of the output. The derived classes only
16 /// implement the steps and so decide about the file format.
17 class Writer : public Object
18 {
19 public:
20    /// @brief Writes all addresses with their persons to a file.
21    ///
22    /// This is the template method. The order is: header, then for each
23    /// address the address, its persons and the end of the address,
24    /// and at last the footer.
25    /// @param[in] path is the path of the file to write.
```

```
26      /// @param[in] man is the manager that holds the data.
27      /// @throws std::out_of_range if a person refers to an address that does not
            exist.
28      /// @throws std::runtime_error if the file cannot be written.
29      void Write(const std::string& path, const IReadOnlyAddressManager& man) const;
30
31  protected:
32      /// @brief Protected default constructor that makes the class abstract.
33      Writer() = default;
34
35      /// @brief Writes the beginning of the file.
36      /// @param[in] out is the stream to write to.
37      virtual void WriteHeader(std::ostream& out) const = 0;
38
39      /// @brief Writes one address.
40      /// @param[in] out is the stream to write to.
41      /// @param[in] address is the address to write.
42      virtual void WriteAddress(std::ostream& out, const Address& address) const = 0;
43
44      /// @brief Writes one person of the current address.
45      /// @param[in] out is the stream to write to.
46      /// @param[in] person is the person to write.
47      virtual void WritePerson(std::ostream& out, const Person& person) const = 0;
48
49      /// @brief Writes the end of the current address block.
50      /// @param[in] out is the stream to write to.
51      virtual void WriteAddressEnd(std::ostream& out) const = 0;
52
53      /// @brief Writes the end of the file.
54      ///
55      /// The default writes nothing and can be overridden in derived classes.
56      /// @param[in] out is the stream to write to.
57      virtual void WriteFooter(std::ostream& out) const {}
58  };
59
60  #endif //WRITER_H
```

Listing 15: ../Writer.cpp

```
1   #include "Writer.h"
2   #include <fstream>
3   #include <stdexcept>
4
5   void Writer::Write(const std::string& path, const IReadOnlyAddressManager& man)
        const
6   {
7       // Check first, so that no half written file is created.
8       for (size_t p = 0; p < man.GetPersonCount(); ++p) {
9           if (man.GetPerson(p).GetAddressID() >= man.GetAddressCount())
10              throw std::out_of_range("Person " + std::to_string(p) +
11                                      " refers to an address that does not exist");
12      }
13
14      std::ofstream out(path);
15      if (!out)
16          throw std::runtime_error("File cannot be written: " + path);
17
18      WriteHeader(out);
19
20      for (size_t a = 0; a < man.GetAddressCount(); ++a) {
21          WriteAddress(out, man.GetAddress(a));
22
23          // Write all persons that refer to this address.
24          for (size_t p = 0; p < man.GetPersonCount(); ++p) {
25              const Person& person = man.GetPerson(p);
26              if (person.GetAddressID() == a)
27                  WritePerson(out, person);
28          }
29          WriteAddressEnd(out);
```

```
30      }
31
32      WriteFooter(out);
33
34      if (!out)
35          throw std::runtime_error("Error while writing: " + path);
36  }
```

Listing 16: ../HtmlWriter.h

```
1  #ifndef HTMLWRITER_H
2  #define HTMLWRITER_H
3  #include "Writer.h"
4
5  /// @author Leander Kieweg
6  /// @date 19.10.2026
7  /// @brief Writes the addresses with their persons as an HTML page.
8  class HtmlWriter : public Writer
9  {
10 protected:
11     /// @brief Writes the HTML head, the title and a horizontal line.
12     /// @param[in] out is the stream to write to.
13     void WriteHeader(std::ostream& out) const override;
14
15     /// @brief Writes an address in italic and opens a new paragraph.
16     /// @param[in] out is the stream to write to.
17     /// @param[in] address is the address to write.
18     void WriteAddress(std::ostream& out, const Address& address) const override;
19
20     /// @brief Writes the name of a person in normal style.
21     /// @param[in] out is the stream to write to.
22     /// @param[in] person is the person to write.
23     void WritePerson(std::ostream& out, const Person& person) const override;
24
25     /// @brief Closes the paragraph of the current address.
26     /// @param[in] out is the stream to write to.
27     void WriteAddressEnd(std::ostream& out) const override;
28
29     /// @brief Closes the body and the HTML page.
30     /// @param[in] out is the stream to write to.
31     void WriteFooter(std::ostream& out) const override;
32 };
33
34 #endif //HTMLWRITER_H
```

Listing 17: ../HtmlWriter.cpp

```
1  #include "HtmlWriter.h"
2
3  void HtmlWriter::WriteHeader(std::ostream& out) const
4  {
5      out << "<!DOCTYPE html>\n"
6          << "<html lang=\"de\">\n"
7          << "<head>\n"
8          << "    <meta charset=\"UTF-8\">\n"
9          << "    <title>Adressen mit zugeordneten Personen</title>\n"
10         << "</head>\n"
11         << "<body>\n"
12         << "<h3>Adressen mit zugeordneten Personen</h3>\n"
13         << "<hr>\n";
14 }
15
16 void HtmlWriter::WriteAddress(std::ostream& out, const Address& address) const
17 {
18     out << "<p>\n"
19         << "<i>" << address.GetZip() << ' ' << address.GetCity() << "<br>\n"
20         << address.GetStreet() << ' ' << address.GetNumber() << "</i><br>\n";
```

```
21 }
22
23 void HtmlWriter::WritePerson(std::ostream& out, const Person& person) const
24 {
25     out << person.GetFirstName() << ' ' << person.GetLastName() << "<br>\n";
26 }
27
28 void HtmlWriter::WriteAddressEnd(std::ostream& out) const
29 {
30     out << "</p>\n";
31 }
32
33 void HtmlWriter::WriteFooter(std::ostream& out) const
34 {
35     out << "</body>\n"
36         << "</html>\n";
37 }
```

Listing 18: ../AsciiWriter.h

```
1  #ifndef ASCIIWRITER_H
2  #define ASCIIWRITER_H
3  #include "Writer.h"
4
5  /// @author Leander Kieweg
6  /// @date 19.10.2026
7  /// @brief Writes the addresses with their persons as plain text.
8  class AsciiWriter : public Writer
9  {
10 protected:
11     /// @brief Writes the title and a line below it.
12     /// @param[in] out is the stream to write to.
13     void WriteHeader(std::ostream& out) const override;
14
15     /// @brief Writes the zip code and city, then the street and number.
16     /// @param[in] out is the stream to write to.
17     /// @param[in] address is the address to write.
18     void WriteAddress(std::ostream& out, const Address& address) const override;
19
20     /// @brief Writes the name of a person, indented by four spaces.
21     /// @param[in] out is the stream to write to.
22     /// @param[in] person is the person to write.
23     void WritePerson(std::ostream& out, const Person& person) const override;
24
25     /// @brief Writes an empty line between two addresses.
26     /// @param[in] out is the stream to write to.
27     void WriteAddressEnd(std::ostream& out) const override;
28 };
29
30 #endif //ASCIIWRITER_H
```

Listing 19: ../AsciiWriter.cpp

```
1  #include "AsciiWriter.h"
2
3  void AsciiWriter::WriteHeader(std::ostream& out) const
4  {
5      const std::string title = "Adressen mit zugeordneten Personen";
6      out << title << '\n'
7          << std::string(title.size(), '-') << '\n';   // line as long as the title
8  }
9
10 void AsciiWriter::WriteAddress(std::ostream& out, const Address& address) const
11 {
12     out << address.GetZip() << ' ' << address.GetCity() << '\n'
13         << address.GetStreet() << ' ' << address.GetNumber() << '\n';
14 }
```

```
15
16 void AsciiWriter::WritePerson(std::ostream& out, const Person& person) const
17 {
18     out << "    " << person.GetFirstName() << ' ' << person.GetLastName() << '\n';
19 }
20
21 void AsciiWriter::WriteAddressEnd(std::ostream& out) const
22 {
23     out << '\n';
24 }
```

Listing 20: ../AddressManager.h

```
1  #ifndef ADDRESSMANAGER_H
2  #define ADDRESSMANAGER_H
3  #include <cstddef>
4  #include <string>
5  #include <vector>
6  #include "Person.h"
7  #include "Address.h"
8  #include "IWriteOnlyAddressManager.h"
9  #include "IReadOnlyAddressManager.h"
10
11 class Reader;
12 class Writer;
13
14 /// @author Leander Kieweg
15 /// @date 19.10.2026
16 /// @brief Manages addresses and persons and links them to each other.
17 ///
18 /// A person refers to its address by an index. The manager reads its data
19 /// with a Reader and writes it with a Writer. Both are passed as pointers,
20 /// so the caller decides which file format is used.
21 class AddressManager : public IWriteOnlyAddressManager, public
       IReadOnlyAddressManager, public Object
22 {
23 public:
24    /// @brief Reads a file with the given reader.
25    /// @param[in] path is the path of the file to read.
26    /// @param[in] rdr is the reader that knows the file format.
27    /// @throws std::invalid_argument if rdr is a null pointer.
28    void ReadFile(const std::string& path, const Reader* rdr);
29
30    /// @brief Writes all addresses with their persons to a file.
31    /// @param[in] path is the path of the file to write.
32    /// @param[in] wrt is the writer that knows the file format.
33    /// @throws std::invalid_argument if wrt is a null pointer.
34    void WriteAddresses(const std::string& path, const Writer* wrt) const;
35
36    /// @brief Access function for an address.
37    /// @param[in] index is the position of the address.
38    /// @returns a constant reference to the address.
39    /// @throws std::out_of_range if the index does not exist.
40    const Address& GetAddress(size_t index) const override;
41
42    /// @brief Access function for a person.
43    /// @param[in] index is the position of the person.
44    /// @returns a constant reference to the person.
45    /// @throws std::out_of_range if the index does not exist.
46    const Person& GetPerson(size_t index) const override;
47
48    /// @brief Access function for the number of addresses.
49    /// @returns the number of stored addresses.
50    size_t GetAddressCount() const override;
51
52    /// @brief Access function for the number of persons.
53    /// @returns the number of stored persons.
54    size_t GetPersonCount() const override;
55
```

```
56      /// @brief Adds a person to the manager.
57      /// @param[in] person is the person to store.
58      void AddPerson(const Person& person) override;
59
60      /// @brief Adds an address to the manager.
61      /// @param[in] address is the address to store.
62      void AddAddress(const Address& address) override;
63
64  private:
65
66      /// @brief All persons in the order they were read.
67      std::vector<Person> mPersons;
68
69      /// @brief All addresses. The position is the index used by the persons.
70      std::vector<Address> mAddresses;
71  };
72
73  #endif
```

Listing 21: ../AddressManager.cpp

```
1   #include "AddressManager.h"
2   #include <stdexcept>
3   #include "Reader.h"
4   #include "Writer.h"
5
6   void AddressManager::ReadFile(const std::string& path, const Reader* rdr)
7   {
8       if (rdr == nullptr)
9           throw std::invalid_argument("Reader is a null pointer");
10      rdr->Read(path, *this);
11  }
12
13  void AddressManager::WriteAddresses(const std::string& path, const Writer* wrt)
        const
14  {
15      if (wrt == nullptr)
16          throw std::invalid_argument("Writer is a null pointer");
17      wrt->Write(path, *this);
18  }
19
20  void AddressManager::AddPerson(const Person& person)
21  {
22      mPersons.push_back(person);
23  }
24
25  void AddressManager::AddAddress(const Address& address)
26  {
27      mAddresses.push_back(address);
28  }
29
30  const Address& AddressManager::GetAddress(size_t index) const
31  {
32      return mAddresses.at(index);   // throws std::out_of_range for a wrong index
33  }
34
35  const Person& AddressManager::GetPerson(size_t index) const
36  {
37      return mPersons.at(index);
38  }
39
40  size_t AddressManager::GetAddressCount() const
41  {
42      return mAddresses.size();
43  }
44
45  size_t AddressManager::GetPersonCount() const
46  {
47      return mPersons.size();
```

```
48 }
```

Listing 22: ../main.cpp

```
1  /// @mainpage Address Management Test Driver
2  /// @brief Test driver for verifying the AddressManagement system.
3  ///
4  /// Tests the reading of person and address files using different Reader
       implementations,
5  /// verifies the integrity of the data stored in the AddressManager, and tests
       exporting
6  /// using AsciiWriter and HtmlWriter. Additionally, all relevant exception cases
7  /// (invalid arguments, runtime errors, out-of-range accesses) are thoroughly
       verified.
8  ///
9  /// @author Leander Kieweg, Julian Rieger
10 /// @date 21.10.2026
11 /// @version 1.0
12
13 #include <iostream>
14 #include <stdexcept>
15 #include <string>
16
17 #include "Address.h"
18 #include "AddressManager.h"
19 #include "AddressReader.h"
20 #include "AsciiWriter.h"
21 #include "HtmlWriter.h"
22 #include "Person.h"
23 #include "PersonReader.h"
24
25 /// @brief Evaluates a test condition and prints a formatted status message to the
        console.
26 ///
27 /// If the condition is true, prints "[SUCCESS]" to standard output (std::cout).
28 /// If false, prints "[FAILED]" to standard error (std::cerr).
29 ///
30 /// @param[in] condition The boolean test result to verify.
31 /// @param[in] testName The descriptive name or purpose of the test.
32 void CheckTest(bool condition, const std::string& testName)
33 {
34     if (condition)
35     {
36         std::cout << "[SUCCESS] " << testName << std::endl;
37     }
38     else
39     {
40         std::cerr << "[FAILED]  " << testName << std::endl;
41     }
42 }
43
44 /// @brief Executes the entire test suite for the address management system.
45 void RunTestDriver();
46
47 /// @brief Program entry point. Runs the test driver.
48 /// @return 0 upon completion.
49 int main()
50 {
51     RunTestDriver();
52     return 0;
53 }
54
55 /// @brief Implementation of the test suite.
56 ///
57 /// Tests the regular workflow (reading files, checking data, writing output)
58 /// followed by comprehensive negative tests verifying expected exceptions:
59 /// - std::invalid_argument when passing nullptr
60 /// - std::runtime_error on non-existent files or corrupt line formats
61 /// - std::out_of_range when accessing invalid indices or missing address
       relations
```

```
62 void RunTestDriver()
63 {
64     AddressManager man;
65     AddressManager noAddresses;
66     AddressManager empty;
67 
68     PersonReader pReader;
69     AddressReader aReader;
70     HtmlWriter hWriter;
71     AsciiWriter aWriter;
72 
73     const std::string personPath{ "../../../pers.txt" };
74     const std::string addressPath{ "../../../adr.txt" };
75     const std::string resultPathHtml{ "../../../result.html" };
76     const std::string resultPathAscii{ "../../../result.txt" };
77 
78     try
79     {
80         man.ReadFile(personPath, &pReader);
81         man.ReadFile(addressPath, &aReader);
82         CheckTest(true, "person and address file are read");
83     }
84     catch (const std::exception&)
85     {
86         CheckTest(false, "person and address file are read");
87     }
88 
89     // Check count of stored elements
90     bool const hasEntries = (man.GetPersonCount() > 0) && (man.GetAddressCount() >
          0);
91     CheckTest(hasEntries, "persons and addresses are stored");
92 
93     // Verify first person
94     const Person& firstPerson = man.GetPerson(0);
95     bool const isDonaldDuck = (firstPerson.GetFirstName() == "Donald") &&
96                               (firstPerson.GetLastName() == "Duck") &&
97                               (firstPerson.GetAddressID() == 0);
98     CheckTest(isDonaldDuck, "first person is Donald Duck with address 0");
99 
100    // Verify first address
101    const Address& firstAddress = man.GetAddress(0);
102    bool const isExpectedAddress = (firstAddress.GetStreet() == "Kellergasse") &&
103                                   (firstAddress.GetNumber() == 3) &&
104                                   (firstAddress.GetZip() == 4287) &&
105                                   (firstAddress.GetCity() == "Radburg");
106    CheckTest(isExpectedAddress, "first address is Kellergasse 3, 4287 Radburg");
107 
108    try
109    {
110        man.WriteAddresses(resultPathAscii, &aWriter);
111        man.WriteAddresses(resultPathHtml, &hWriter);
112        CheckTest(true, "ascii and html file are written");
113    }
114    catch (const std::exception&)
115    {
116        CheckTest(false, "ascii and html file are written");
117    }
118 
119    // Exception Handling Tests: std::invalid_argument
120    try
121    {
122        man.ReadFile(personPath, nullptr);
123        CheckTest(false, "null reader -> invalid_argument");
124    }
125    catch (const std::invalid_argument&)
126    {
127        CheckTest(true, "null reader -> invalid_argument");
128    }
129 
130    try
```

```
131     {
132         man.WriteAddresses(resultPathAscii, nullptr);
133         CheckTest(false, "null writer -> invalid_argument");
134     }
135     catch (const std::invalid_argument&)
136     {
137         CheckTest(true, "null writer -> invalid_argument");
138     }
139
140     // Exception Handling Tests: std::runtime_error
141     try
142     {
143         man.ReadFile("no_such_file.txt", &pReader);
144         CheckTest(false, "missing file -> runtime_error");
145     }
146     catch (const std::runtime_error&)
147     {
148         CheckTest(true, "missing file -> runtime_error");
149     }
150
151     try
152     {
153         // Parsing person file with address reader -> mismatching format
154         man.ReadFile(personPath, &aReader);
155         CheckTest(false, "wrong line format -> runtime_error");
156     }
157     catch (const std::runtime_error&)
158     {
159         CheckTest(true, "wrong line format -> runtime_error");
160     }
161
162     // Exception Handling Tests: std::out_of_range
163     try
164     {
165         man.GetAddress(man.GetAddressCount());
166         CheckTest(false, "wrong address index -> out_of_range");
167     }
168     catch (const std::out_of_range&)
169     {
170         CheckTest(true, "wrong address index -> out_of_range");
171     }
172
173     try
174     {
175         empty.GetPerson(0);
176         CheckTest(false, "wrong person index -> out_of_range");
177     }
178     catch (const std::out_of_range&)
179     {
180         CheckTest(true, "wrong person index -> out_of_range");
181     }
182
183     try
184     {
185         // Person references address ID that was never loaded
186         noAddresses.ReadFile(personPath, &pReader);
187         noAddresses.WriteAddresses(resultPathAscii, &aWriter);
188         CheckTest(false, "person without address -> out_of_range");
189     }
190     catch (const std::out_of_range&)
191     {
192         CheckTest(true, "person without address -> out_of_range");
193     }
194 }
```